

CRDs & Operators

Kubernetes Production · Lesson 15

Built from the official Kubernetes documentation

`kubernetes.io/docs`

Learning Objectives

By the end of this lesson you will be able to:

- Explain that the Kubernetes API is **declarative** and **extensible**
- Recognise the **extension points**, including **CRI / CNI / CSI**
- Define a **custom resource** and when to use one vs a ConfigMap
- Compare the two API-extension paths: **CRD** vs **API aggregation**
- Author a **CustomResourceDefinition** (apiextensions.k8s.io/v1)
- Describe the **controller control loop** and the **Operator pattern**
- Install an **operator** (manifests / Helm) and drive it with custom resources

Reference: *Extending Kubernetes; Custom Resources; Operator pattern* — kubernetes.io/docs

15.1 Extending Kubernetes

A Declarative, Extensible API

Kubernetes is built around a **declarative API**: you describe **desired state** in objects, and **controllers** continuously reconcile reality to match it.

The platform is designed to be **extended** without forking Kubernetes:

- **Client extensions** — `kubectl` / credential plugins
- **API-access** — authentication, authorization, **admission webhooks**
- **API additions** — **CRDs** and the **aggregation layer**
- **Infrastructure** — **CRI**, **CNI**, **CSI**, device plugins
- **Scheduling** — custom schedulers / extenders
- **Controllers** — watch state and reconcile

This lesson focuses on the **API-additions** point: **custom resources**.

The Extension Points

Extension point	What it customises
kubectl / credential plugins	Client subcommands, auth
Authn / Authz	Who calls the API, and what they may do
Admission webhooks	Block, mutate, or validate requests
CRDs / aggregation layer	Add new API kinds
CRI	Container runtime interface
CNI	Pod network interface
CSI	Container storage interface
Scheduling / controllers	Placement and reconciliation

The **webhook** and **controller** patterns recur across many of these points.

Infrastructure Interfaces: CRI / CNI / CSI

The CKA expects you to **recognise** the three infrastructure interfaces — stable APIs that let a cluster swap implementations:

- **CRI — Container Runtime Interface**: how the **kubelet** talks to the container runtime (e.g. **containerd**, **CRI-O**)
- **CNI — Container Network Interface**: pluggable **Pod networking** (e.g. **Calico**, **Cilium**, **Flannel**)
- **CSI — Container Storage Interface**: pluggable **storage backends** mounted into Pods

Each is an **interface** behind which vendors provide plugins — the control plane stays the same while runtime / network / storage are swapped.

15.2 Custom Resources

What Is a Custom Resource?

A **custom resource** is *"an extension of the Kubernetes API that is not necessarily available in a default Kubernetes installation."*

- It is an **endpoint in the Kubernetes API** that stores a collection of objects
- It can **appear and disappear dynamically** in a running cluster
- You use it with `kubectl` **just like built-in resources** (Pods, Deployments)

On its own, a custom resource only **stores structured data**. Combined with a **custom controller**, it becomes a true **declarative API** that *acts*.

- Caution: don't use a custom resource as **data storage** for application or end-user data — that couples your app too tightly to the API server.

ConfigMap vs Custom Resource

Use a ConfigMap when...	Use a Custom Resource when...
There is an existing config-file format (e.g. <code>mysql.cnf</code>)	You want top-level <code>kubectl get <kind></code> support
A program in a Pod reads the file to configure itself	You want API conventions: <code>.spec</code> , <code>.status</code> , <code>.metadata</code>
Consumers prefer file/env access	You want automation that watches and reconciles
Rolling updates come from the Deployment	The object abstracts a collection of controlled resources

Reach for a **custom resource** when the object is a first-class **API** that something should act on — not just a config blob.

15.3 Two Ways to Add a Custom API

CRD vs API Aggregation

	CustomResourceDefinition	API Aggregation
Programming	None — declarative YAML	Required — build an apiserver
What runs	The existing kube-apiserver	A subordinate apiserver behind a proxy
Flexibility	Less (generic impl.)	More — storage, version conversion
Typical use	The common path	Complex APIs needing fine control

"Kubernetes provides these two options ... so that neither ease of use nor flexibility is compromised."

- **CRD is the no-code, common path** — and the focus of the CKA.

15.4 CustomResourceDefinition

The CRD Object

A **CustomResourceDefinition** is itself a Kubernetes object that **registers a new kind**.

- `apiVersion: apiextensions.k8s.io/v1`
- `kind: CustomResourceDefinition`
- `metadata.name` **must** be `<plural>.<group>`
(e.g. `crontabs.stable.example.com`)

Creating it makes the API server expose a **new RESTful endpoint** per served version:

```
/apis/stable.example.com/v1/namespaces/*/crontabs/...
```

A CRD is a **cluster-scoped** object — even when the custom resources it defines are **Namespaced**.

CRD Fields

Field	Meaning
<code>spec.group</code>	API group: <code>/apis/<group>/<version></code>
<code>spec.versions[]</code>	Versions; each has <code>served</code> + <code>storage</code>
<code>served</code> / <code>storage</code>	<code>served</code> = reachable; exactly one is <code>storage: true</code>
<code>schema.openAPIV3Schema</code>	Structural schema (mandatory in <code>v1</code>)
<code>spec.scope</code>	<code>Namespaced</code> or <code>Cluster</code>
<code>spec.names.plural</code>	URL plural (<code>crontabs</code>)
<code>spec.names.singular</code>	CLI alias / display (<code>crontab</code>)
<code>spec.names.kind</code>	CamelCase kind used in manifests (<code>CronTab</code>)
<code>spec.names.shortNames</code>	CLI short aliases (<code>ct</code>)

Full CRD Example (CronTab)

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: crontabs.stable.example.com # <plural>.<group>
spec:
  group: stable.example.com
  versions:
    - name: v1
      served: true # exposed via the API
      storage: true # exactly one storage version
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                cronSpec: { type: string }
                image: { type: string }
                replicas: { type: integer }
  scope: Namespaced
  names:
    plural: crontabs
    singular: crontab
    kind: CronTab
    shortNames: ["ct"]
```

Creating a Custom Resource

Once the CRD is applied, create **objects of the new kind**:

```
apiVersion: stable.example.com/v1    # <group>/<version>
kind: CronTab                        # matches spec.names.kind
metadata:
  name: my-new-cron-object
spec:
  cronSpec: "* * * * */5"
  image: my-awesome-cron-image
```

```
kubectl apply -f crd.yaml           # register the new kind
kubectl apply -f crontab.yaml       # create an instance
kubectl get crontabs                # or: kubectl get ct
```

apiVersion + **kind** must match what the CRD declared.

Printer Columns & Subresources

CRDs can be enriched beyond plain storage:

- **additionalPrinterColumns** — extra columns in `kubectl get <kind>` (e.g. show `spec.replicas` or an age column)
- **Subresources**:
 - **status** — a `/status` endpoint so controllers update status independently of users editing `spec`
 - **scale** — a `/scale` endpoint so `kubectl scale` and the **HPA** can drive the resource

Structural schema is mandatory in `v1`: every field needs a `type` at the root, each object property, and each array item.

CRD Commands

```
kubectl get crd                # all CRDs (cluster-scoped)
kubectl describe crd crontabs.stable.example.com
kubectl get crontabs -A        # custom objects, all namespaces
kubectl explain crontab.spec   # schema of the new kind
kubectl api-resources | grep stable # find kinds from a CRD's group
```

CKA flow: **get crd** to spot extensions → **api-resources** to find the new kinds → **explain** to learn their schema.

15.5 Controllers & Operators

The Controller Control Loop

Every Kubernetes controller runs a **control loop**:

1. **Observe** — read the object's `.spec` (desired state)
2. **Diff** — compare desired state with **actual** cluster state
3. **Act** — make changes to reconcile, then update `.status`



This loop is the **core pattern** of Kubernetes — built-ins and custom controllers work the same way.

The Operator Pattern

Operators are *"software extensions to Kubernetes that make use of custom resources to manage applications and their components" and "follow Kubernetes principles, notably the control loop."*

Operator = CRD(s) + a custom controller

- The **CRD** defines the API (e.g. a `SampleDB` kind)
- The **controller** runs the loop and **encodes operational knowledge** — how a human operator would deploy, back up, upgrade and recover the app
- The controller usually runs **outside the control plane**, as a **Deployment**

You declare *what you want* (a custom resource); the operator makes reality match and keeps it healthy.

What Operators Automate

Operators capture the expertise of a human operator. From the docs, they can:

- **Deploy** an application on demand
- **Take and restore backups** of application state
- **Handle upgrades** — including schema and config changes
- **Publish services** for apps that don't speak the Kubernetes API
- **Simulate failure** to test resilience
- Manage **leader election** for distributed apps

Real examples: **Prometheus Operator**, **etcd** / database operators (PostgreSQL, MySQL), **cert-manager**.

Installing an Operator

*"The most common way to deploy an operator is to add the **Custom Resource Definition** and its associated **Controller** to your cluster."*

Installing an operator (via **manifests** or a **Helm chart** — see Lesson 12) typically brings in:

- the **CRDs** (the new custom kinds)
- the **controller Deployment** (runs the reconcile loop)
- the **RBAC** (ServiceAccount + Role/ClusterRole bindings)

Then you **drive** it with custom resources:

```
kubectl get SampleDB           # find configured instances
kubectl edit SampleDB/example-database # change settings → operator reconciles
```

Key Takeaways

- The Kubernetes API is **declarative** and **extensible**; **CRI / CNI / CSI** are the infrastructure interfaces (runtime / network / storage)
- A **custom resource** extends the API and is used with `kubectl` like built-ins; alone it only stores data — a **controller** makes it act
- Two ways to add an API: **CRD** (declarative, no code, common path) vs **API aggregation** (run your own apiserver, more control)
- A CRD is `apiextensions.k8s.io/v1`, named `<plural>.<group>`, with **exactly one** `storage: true` version; CRDs are **cluster-scoped**
- **Operator = CRD(s) + custom controller** running the **observe → diff → act** loop
- Install operators with **manifests / Helm** → CRDs + Deployment + RBAC; inspect with `kubectl get crd`, `api-resources`, `explain`

End of Lesson 15

Next: Lesson 16 — Node Maintenance & Scaling

```
kubectl get crd · kubectl api-resources · kubectl explain <kind> · kubectl apply -f crd.yaml
```